

Method Hiding in C# with Examples

In this article, I am going to discuss **Method Hiding in C#** with Examples. Please read our previous article, where we discussed **Method Overriding in C#** with Examples. At the end of this article, you will understand what exactly Method Hiding is and when and how to use Method Hiding in C# with Multiple Examples.

What is Method Hiding in C#?

Method Overriding is an approach of re-implementing the parent class methods under the child class exactly with the same signature (same name and same parameters).

Method Hiding/Shadowing is also an approach of re-implementing the parent class methods under the child class exactly with the same signature (same name and same parameters).

How we can Re-Implement a Parent Method in the Child Class in C#?

We can re-implement the parent class methods under the child classes in two different approaches. They are as follows

1. **Method Overriding**
2. **Method Hiding**

Then what are the differences between them, let us understand.

In Method Overriding, the child classes re-implement their parent class methods which are declared as virtual. That means here, the child classes re-implement the parent class methods with the permission of the parent class because here in the parent class the method is declared as virtual giving permission to the child classes for overriding the methods using the override modifier.

In Method Hiding/Shadowing, the child classes can re-implement any method of its parent class methods even if they are not declared as virtual. That means here the child class re-implements the parent class methods without taking any permission from its parent.

How to Implement Method Hiding/Shadowing in C#?

Please have a look at the following image to understand the syntax of Method Hiding/Shadowing in C#. It does not matter whether the parent class method is virtual or not. We can hide both virtual and non-virtual methods under the child class. Again, we can hide the method in the child class in two ways i.e. by using the new keyword and also, without using the new keyword. If we are not using the new keyword then we will get a warning and the reason for the warning we will discuss later part of this article.

```

public class Parent
{
    public virtual void Show() { }
    public void Display() { }
}

public class Child : Parent
{
    //Method Hiding using new keyword
    public new void Show() { }

    //Method Hiding without using new keyword
    //Getting one warning
    public void Display() { }
}

```

When we use the new keyword to hide a Parent Class Methods under the child class, then it is called Method Hiding/Shadowing in C#. Using the new keyword for re-implementing the Parent Class Methods under the child class is optional.

Example to Understand Method Hiding/Shadowing in C#:

Please have look at the following example. Here, inside the Parent class, we have declared two methods i.e. Show and Display. The Show method is declared as virtual and Display is not declared as virtual. And then the Child class is inheriting from the Parent class. That means the Child class is now also having the parent class methods. And when we create the instance of the Child class and invoke the methods, then the methods are going to be executed from the Parent class. This is the concept of **Inheritance** and we have already discussed this concept.

```

using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Show()
        {
            Console.WriteLine("Parent Class Show Method");
        }
        public void Display()
        {
            Console.WriteLine("Parent Class Display Method");
        }
    }
    public class Child : Parent
    {

```

```

    }
    class Program
    {
        static void Main(string[] args)
        {
            Child obj = new Child();
            obj.Show();
            obj.Display();
            Console.ReadKey();
        }
    }
}

```

Output:

```

Parent Class Show Method
Parent Class Display Method

```

Now, as a child class user, we don't like the above output. We want our own versions of the above two methods inside the Child class. If you look at the Show method, then it is declared as virtual inside the Parent class, so we can override this virtual method inside the Child class by using the override modifier. But we cannot override the Display method inside the Child class as it is not declared as virtual inside the Parent class. But we want to re-implement the method. In that case, we need to re-implement the Parent Class Display Method using the new keyword inside the Child class which is nothing but Method Hiding/Shadowing in C#. The complete example code is given below.

```

using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Show()
        {
            Console.WriteLine("Parent Class Show Method");
        }
        public void Display()
        {
            Console.WriteLine("Parent Class Display Method");
        }
    }
    public class Child : Parent
    {
        //Method Overriding
        public override void Show()
        {
            Console.WriteLine("Child Class Show Method");
        }
        //Method Hiding/Shadowing
        public new void Display()

```

```

    {
        Console.WriteLine("Child Class Display Method");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Child obj = new Child();
        obj.Show();
        obj.Display();
        Console.ReadKey();
    }
}

```

Output:

```

Child Class Show Method
Child Class Display Method

```

So, here, you can observe both Method Overriding and Method Hiding doing the same thing. That is re-implementing the Parent class methods under the Child class. Then what are the differences between them? With method overriding, you can re-implement only virtual methods. On the other hand, with Method Hiding, you can re-implement any methods.

For a better understanding, please have a look at the following example. Here, inside the Parent class, we have declared defined two virtual methods and two non-virtual methods. Inside the Child class, we are re-implementing both virtual and non-virtual methods using both Method Overriding and Method Hiding. Here, you will get a compile-time error while you are trying to re-implement the non-virtual method using the override modifier but you will not get an error when you are re-implementing the virtual and non-virtual method using the new keyword.

```

using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Method1()
        {
            Console.WriteLine("Parent Class Method1 Method");
        }
        public void Method2()
        {
            Console.WriteLine("Parent Class Method2 Method");
        }
        public virtual void Method3()
        {
            Console.WriteLine("Parent Class Method3 Method");
        }
    }
}

```

```

    public void Method4()
    {
        Console.WriteLine("Parent Class Method4 Method");
    }
}
public class Child : Parent
{
    //Overriding Virtual Method
    //Method Overriding
    public override void Method1()
    {
        Console.WriteLine("Child Class Method1 Method");
    }
    //Overriding Non-Virtual Method
    //Not Possible. Compile Time Error
    public override void Method2()
    {
        Console.WriteLine("Child Class Method2 Method");
    }

    //Method Hiding/Shadowing Virtual Method
    public new void Method3()
    {
        Console.WriteLine("Child Class Method3 Method");
    }
    //Method Hiding/Shadowing Non-Virtual Method
    public new void Method4()
    {
        Console.WriteLine("Child Class Method4 Method");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Child obj = new Child();
        obj.Method1();
        obj.Method2();
        obj.Method3();
        obj.Method4();
        Console.ReadKey();
    }
}

```

When you try to run the above code, you will get the following error and this makes sense because we cannot override the non-virtual method inside the Child class.

	Code	Description
	CS0506	'Child.Method2()': cannot override inherited member 'Parent.Method2()' because it is not marked virtual, abstract, or override

Why we are using the new keyword in Method Hiding?

The new keyword explicitly tells us that you are hiding the base class or parent class members inside the child class. Even if you are not using the new keyword, then also you can re-implement the method under the child, but in that case, you will get a warning from the compiler as shown in the below image.

```
public class Parent
{
    public void Display(){}
}
public class Child : Parent
{
    //Method Hiding/Shadowing
    public void Display() {}
}
```

 void Child.Display()
'Child.Display()' hides inherited member 'Parent.Display()'. Use the new keyword if hiding was intended.
[Show potential fixes](#) (Alt+Enter or Ctrl+.)

Here, you can see that the compiler gives us one warning that the Child class Display method hides the inherited Parent class Display method. In the second message, it is saying that if you want to hide the member, then please use the new keyword.

Let us try to understand the warning in more detail. Today, we intentionally defined the method Display inside the Child class which is already present inside the Parent class. Tomorrow, there might be a chance while you are defining the Child class, you might forget that there is a method called Display defined in the Parent class. At that time, the compiler will warn you. What is the warning? The compiler will warn you that already this method is defined in the Parent class and this method will hide the Parent class method from now onwards. So, if you are mistaken better change the name of the method, or if hiding was your intention, then please use the new keyword.

So, when we use the new keyword, it is just a piece of information to the compiler that the programmer intentionally defined a method with the same name and same signature as the parent class method. So, if you removed the new keyword there is no difference in the program execution only you will get a warning. That is the only use of the new keyword in this context.

How to Invoke the Parent class Methods from the Child Class re-implemented methods?

In C#, we can re-implement the Parent class methods under the child class in two ways. They are as follows:

1. **Method Overriding**
2. **Method Hiding**

In the first case, with permission, and in the second case, without permission. In Method Overriding and Method Hiding, after performing the re-implementation, a call to the method by using the child class instance always invokes the local methods i.e. child class methods. For a better understanding, please have a look at the following example. Here, we create an instance of the Child class and then invoke both the methods, and in this case, both the methods are going to be executed from the Child class only.

```
using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Show()
        {
            Console.WriteLine("Parent Class Show Method");
        }
        public void Display()
        {
            Console.WriteLine("Parent Class Display Method");
        }
    }
    public class Child : Parent
    {
        //Method Overriding
        public override void Show()
        {
            Console.WriteLine("Child Class Show Method");
        }

        //Method Hiding/Shadowing
        public new void Display()
        {
            Console.WriteLine("Child Class Display Method");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Child obj = new Child();
            obj.Show();
            obj.Display();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Child Class Show Method
Child Class Display Method
```

After re-implementing the Parent classes methods under the child classes, the child class instance will start calling the local methods only i.e. the re-implemented methods, but if required in any case we can also call the Parent class methods from the child classes by using two approaches.

1. **Using the Parent Class Instance**
2. **Using the base keyword**

So, by creating the parent class object under the child class, we can call the parent class methods from the child class, or by using the base keyword, we can also call parent class methods from the child class, but the keyword like this and base cannot be used under the static block.

For a better understanding, please have a look at the following example. Here, you can see, inside the Overriding and Hiding method, we invoke the parent class methods using both approaches. You can invoke any of the parent methods. It is not like you can only invoke the Parent class Show method from the Overriding Show method, you can also invoke the Display method.

```
using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Show()
        {
            Console.WriteLine("Parent Class Show Method");
        }
        public void Display()
        {
            Console.WriteLine("Parent Class Display Method");
        }
    }
    public class Child : Parent
    {
        //Method Overriding
        public override void Show()
        {
            //Using Parent class instance to Invoke the Parent Methods
            Parent parent = new Parent();
            parent.Show();
            //Using base keyword to invoke the Parent method
            base.Display();
            Console.WriteLine("Child Class Show Method");
        }

        //Method Hiding/Shadowing
        public new void Display()
        {
            //Using Parent class instance to Invoke the Parent Methods
```

```

        Parent parent = new Parent();
        parent.Display();
        //Using base keyword to invoke the Parent method
        base.Show();
        Console.WriteLine("Child Class Display Method");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Child obj = new Child();
        obj.Show();
        obj.Display();
        Console.ReadKey();
    }
}
}

```

Output:

```

Parent Class Show Method
Parent Class Display Method
Child Class Show Method
Parent Class Display Method
Parent Class Show Method
Child Class Display Method

```

Differences Between Method Overriding and Method Hiding in C#:

As of now, we have not found any major differences between Method Overriding and Method Hiding in C#. There are some differences in terms of implementation, but from the execution point of view, when we invoked the methods using the child class instance, then both the methods are going to be executed from the child class. Then what is the major difference between them, let us try to understand.

Can we create a Parent class reference variable using the Child class instance in C#? Yes, it is possible and we have already discussed this in our Inheritance article. So, a Parent class reference variable can hold the child class object reference.

For a better understanding, please have a look at the following example. Here, we are creating an instance of the Child class and storing that instance reference in the Parent class reference variable, and then invoking the two methods.

```

using System;
namespace MethodHiding
{
    public class Parent
    {
        public virtual void Show()
    }
}

```

```

    {
        Console.WriteLine("Parent Class Show Method");
    }
    public void Display()
    {
        Console.WriteLine("Parent Class Display Method");
    }
}
public class Child : Parent
{
    //Method Overriding
    public override void Show()
    {
        Console.WriteLine("Child Class Show Method");
    }

    //Method Hiding/Shadowing
    public new void Display()
    {
        Console.WriteLine("Child Class Display Method");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Parent obj = new Child();
        obj.Show();
        obj.Display();

        Console.ReadKey();
    }
}

```

Output:

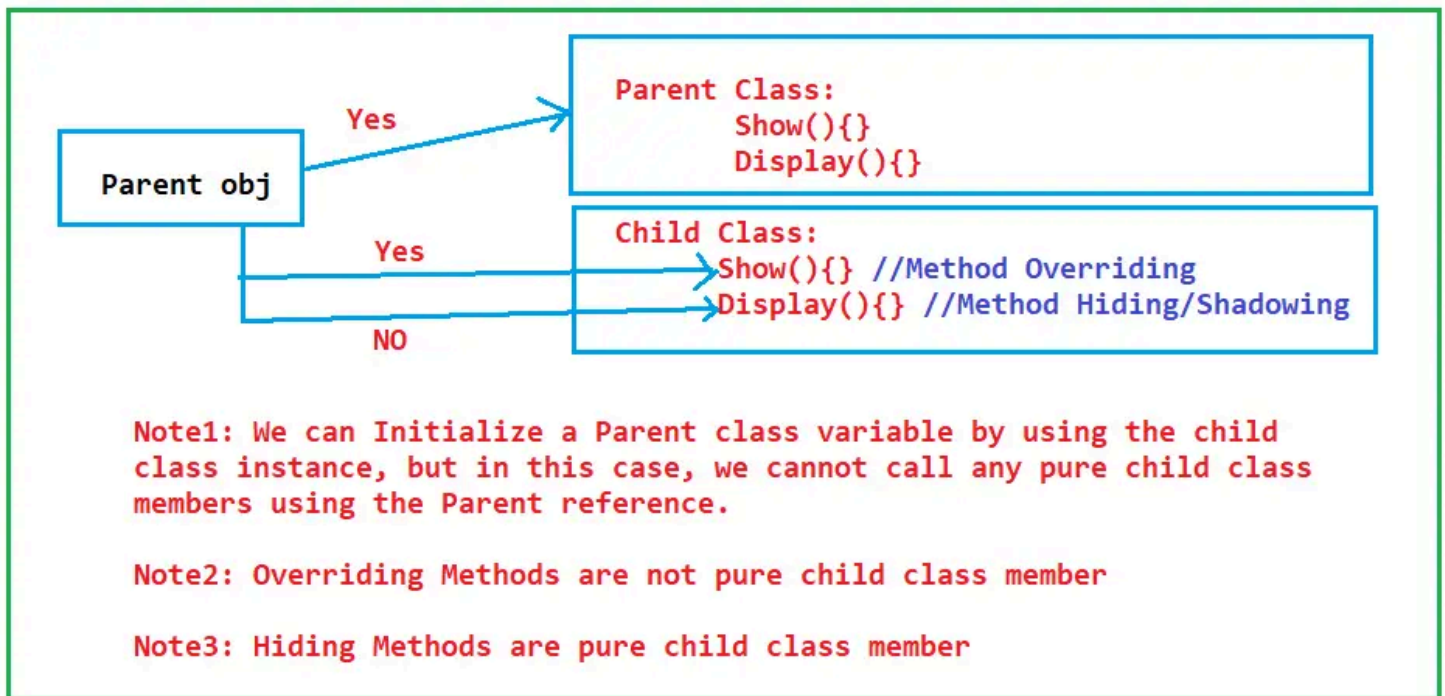
```

Child Class Show Method
Parent Class Display Method

```

You can see in the above output that the Show method (Overriding Method) is executed from the Child class while the Display method (Hiding Method) is executed from the Parent class. Why?

The point that you need to remember is that a parent class reference variable even if created by using the child class instance, cannot access any members which are purely defined inside the child class but can call the overriding methods of the child class because overridden members are not considered as pure child class members, but the members which are re-implemented inside the child class by using the approach of hiding are considered as pure child class members and cannot be accessed by Parent reference variable. For a better understanding, please have a look at the following diagram.



Using Parent obj, we can access the parent class members and if it is created using the child class instance, then we can access the overriding members of the Child class.

Here, the Show method is an overriding method and overriding methods are not pure child class methods. The Show method is defined in the Parent class and again re-defined inside the Child class, and hence we can invoke the Show method using the Parent reference variable and when we invoke the Show method using the Parent reference variable, as it holds the object of the child class, so, the method is going to be executed from the Child class only.

So, the point that you need to remember is in the case of method overriding, the Parent class gives permission to the child classes to override the method. And when the child class re-implements the method, the parent class identifies them or the Parent class recognizes them. As the Parent class recognizes. So, it is able to call the child class overriding methods.

But this is not the case with Method Hiding/Shadowing. In Method Hiding, without taking the Parent's permission, the Child class started re-implementing the methods. As the child does not take any permission from the Parent, so, the Parent reference variable does not recognize them and hence the Parent reference variable cannot access them. So, in this case, the Parent class method is going to invoke. So, hiding a Parent Member inside the child class is considered a pure child class member.

This is the major difference between Method Overriding and Method Hiding in C#. So, if a Parent class reference variable is created using the Child class object reference, then using that Parent reference we can call the Child class overriding methods but we cannot call the child class hiding methods.

In the next article, I am going to discuss [Partial Class and Partial Methods in C#](#) with Examples. Here, in this article, I try to explain What exactly Method Hiding is and how to implement **Method Hiding in C#** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Method Hiding in C# with Examples article.